

LAB EXERCISE: 2

Intelligent Question Answering and Chatbot System

Student Name	Aditi S Yaranal	USN	1GA23AI003
Semester / Department	7th Semester / AIML	Date	8 October 2026
Institution	Global Academy of Technology	Course	AML23702

Aim / Objective

To design and implement a question answering and chatbot system combining information-retrieval-based factoid QA, structured knowledge-based QA and simple dialogue management, and evaluate its answers and conversational context.

Expected Outcomes Addressed

- Retrieve relevant documents and evidence sentences using TF-IDF and cosine similarity.
- Extract factoid answer spans such as people, locations and dates.
- Identify disease entities and relations, then query a structured CSV knowledge source.
- Maintain a conversation frame and resolve simple pronoun follow-ups.
- Evaluate exact match, token F1, expected-fact coverage, intent and entity-state correctness.

Tools & Technologies Used

Category	Tool	Purpose
Backend	Python 3.12 / FastAPI 0.115	Local API and validation
NLP / retrieval	scikit-learn 1.5.2 / regular expressions	TF-IDF, cosine ranking and span extraction
Frontend	React 18 / Vite 5 / Tailwind CSS 3	Interactive QA, chat and evaluation
Data	Text, CSV and JSON	Corpus, structured facts and test questions
Verification	unittest / FastAPI TestClient	Regression and API checks

Dataset / Corpus Description

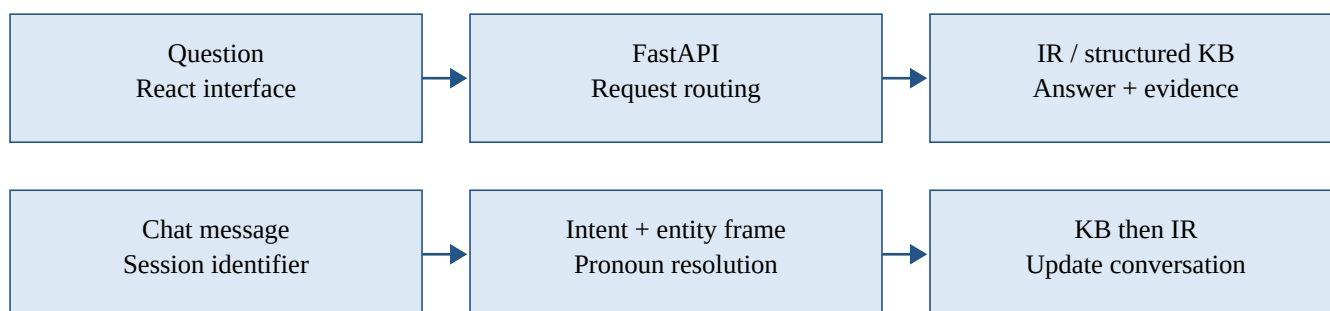
The repository supplies eight short English text documents spanning science, history, geography, technology and education. These are a small educational corpus; they are not a continuously updated external knowledge source. Text is split into sentences and indexed with English stop-word removal.

The healthcare CSV has 15 disease entities, aliases and six relations: category, symptoms, causes, treatment, specialist and prevention. CSV rows are loaded into an entity-to-relations dictionary. This demonstrates structured querying without requiring a database server.

Healthcare values are educational sample data without per-row medical provenance. Responses explicitly identify them as sample KB entries. They must not be used for diagnosis or treatment advice.

Evaluation partition	Size	Reference format
IR questions	17	Full expected answer span
KB questions	15	Expected fact phrase within a longer stored value
Dialogue sequence	7 turns	Intent on all turns; entity state on four turns

System Architecture / Pipeline Design



IR pipeline: normalize and expand common abbreviations; rank documents with TF-IDF; rank candidate sentences; apply event-sensitive begin/end ranking; extract an answer; return its actual evidence document. The displayed confidence is a heuristic score, not a calibrated probability.

Knowledge pipeline: longest alias with word boundaries; relation keyword matching; entity/relation lookup; template-based answer. Unsupported relations return no structured answer rather than an unrelated category.

Backend Implementation

6.1 Approach / Algorithm

IR extraction uses WH-question type, date expressions, proper-name candidates and grammatical patterns. Agent phrases preserve compound founders such as Bill Gates and Paul Allen; role questions can extract the sentence subject. Evidence remains inspectable.

The dialogue frame stores current_entity, previous_question, previous_answer, detected_intent and the last six message entries. Greetings, help and goodbye have explicit handling. A complete goodbye expression is required, so a question containing the word end remains a question. Pronouns can reuse a known healthcare entity; an unrelated IR topic clears that entity.

6.2 Key Code Snippets

```
if re.search(r"(?<!\w)" + re.escape(alias) + r"(?!\\w)",
            question.lower()):
    return canonical
```

Word boundaries avoid matching the flu alias inside influence.

```
best_doc = next(d for d in retrieved_docs
                if d["doc_id"] == candidate_sentences[0]["doc_id"])
best_sentence_text = candidate_sentences[0]["text"]
```

The answer source follows the selected sentence, even if it comes from a document other than the top-ranked passage.

```
resolved_message = self.resolve_coreference(message, session)
kb_result = self.knowledge_service.query(
    resolved_message, context_entity=None)
```

Context is introduced through an explicit pronoun resolution step, avoiding unconditional reuse of the previous entity.

Frontend / UI Implementation

7.1 Framework Choice

React and React Router provide five pages: Home, IR-Based QA, Knowledge-Based QA, Chat and Evaluation. Vite proxies local API calls to FastAPI; Tailwind provides the existing dark interface.

7.2 Interface Features

The interface exposes retrieved evidence, pipeline stages, structured records, source badges, chat reset and live evaluation tables. A per-tab session identifier separates conversations. Reset is disabled while an answer is loading.

Sample Input & Output

Input	Observed output / behavior
Who founded Microsoft?	Bill Gates and Paul Allen
Who is the CEO of Microsoft?	Satya Nadella (from the supplied static document)
When did World War II end in Europe?	May 8, 1945
Who created Python?	Guido van Rossum
What are the symptoms of Diabetes?	Returns the symptoms value from the sample CSV; current entity becomes Diabetes.
What is its treatment?	Resolves its to Diabetes and queries treatment.
Who created Python? (after Diabetes)	Routes to document retrieval and clears the healthcare entity.

Results (Deliverable-Specific Outputs)

- Working prototype: FastAPI backend and React interface, fully local with no LLM/API key requirement.
- Sample QA set: backend/data/evaluation_dataset.json; 17 IR questions, 15 KB questions and seven dialogue turns.
- Dialogue frame and flow: documented in docs/dialogue_flow.md.
- Evaluation summary: saved before/after JSON, regression results and the live Evaluation page.
- Submission: source code, reproducible setup, this PDF and the report builder.

Evaluation / Analysis

Measure	Before review	After review
IR exact match (17)	82.4% (14/17)	100% (17/17)
Mean token F1 (17)	85.7%	100%
KB expected-fact coverage (15)	100% (original metric)	100% (complete phrase required)
Dialogue intent (7)	100%	100%
Dialogue entity state (4)	Not evaluated	100%
Regression / API checks	Not supplied	9 / 9 passed

These are development-set results, including cases used to guide fixes. They do not establish general accuracy. The KB score measures expected phrase coverage, not exact match of the entire answer or medical validity. No held-out benchmark or statistical generalization claim is made.

Evaluation / Analysis (continued)

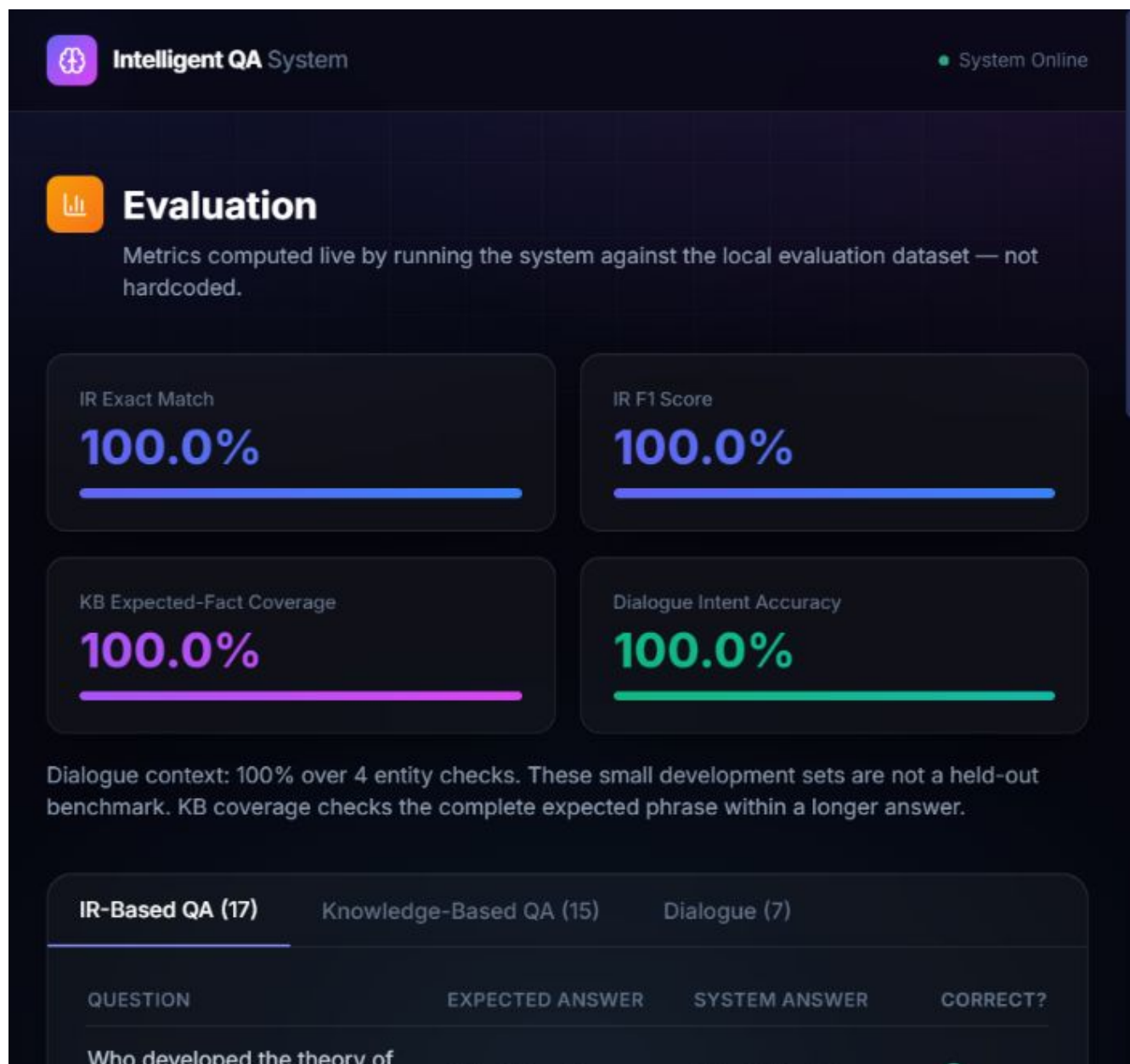


Figure 1. Live evaluation page captured on 8 October 2026.

Challenges Faced & Solutions

Incorrect span selection was corrected using agent/subject patterns and event-sensitive ranking. Source attribution now follows the evidence sentence. Alias boundaries prevent accidental disease matches, and unsupported relations no longer default silently to a category. Chat topic changes no longer inherit the previous healthcare entity.

Conclusion

The prototype satisfies the three core implementation areas of Exercise 2: IR-based factoid QA, structured knowledge QA and basic dialogue management. It presents evidence and measures results transparently. Rule-based extraction remains sensitive to wording, multi-date sentences and unsupported questions; future work should add held-out evaluation, stronger relation parsing and sourced knowledge data.

The frontend production build passed. Compatible dependency updates were applied, but npm audit still reports 11 dependency advisories (five moderate, six high). Resolving the remaining reports requires a separately tested major-version migration. Use the application as a local academic demo; production hardening is outside this exercise.

Project Repository (GitHub Link)

Field	Submission details
Repository	https://github.com/Aditisy/qa-chatbot-system
Branch	main
Student / USN	Aditi S Yaranal / 1GA23AI003
Submission date	8 October 2026

Reproduction Commands (Windows PowerShell)

```
py -3.12 -m venv .venv
.\.venv\Scripts\python -m pip install -r backend/requirements.txt
.\.venv\Scripts\python -m uvicorn app.main:app --app-dir backend
# In a second terminal:
cd frontend
npm ci
npm run dev -- --host 127.0.0.1
```

Open <http://127.0.0.1:5173>. Run tests from the repository root after installing backend/requirements-dev.txt and setting PYTHONPATH to backend. Runtime files and dependencies are excluded from Git.

References

- Lab Exercise with Rubrics (2).pdf: Exercise 2 objective, outcomes, deliverables and assessment rubric.
- ANLP_Lab1_report013.pdf: supplied report format; section structure adapted to Exercise 2, matching the Lab 3 submission style.
- Repository source, corpus and evaluation_dataset.json: <https://github.com/Aditisy/qa-chatbot-system>
- TF-IDF and cosine similarity are implemented with scikit-learn; the application uses FastAPI, React, Vite and Tailwind CSS.

ADVANCE NLP (AML23702)

LAB EXERCISE EVALUATION SHEET

Student Name	Aditi S Yaranal
USN	1GA23AI003
Lab Exercise No.	2
Experiment Title	Intelligent Question Answering and Chatbot System
Semester / Department	7th Semester / AIML
Institution	Global Academy of Technology

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
	Total	50	

Overall Comments

Date of Submission	8 October 2026
Signature of Faculty	